

The next step is to define the models you need. Let's write them in `app_models.py`. Each track can belong to multiple albums, and each album contains multiple tracks. The only note-worthy line is the assignment of 1 to the `_p_changed` variable, to tell ZODB that a mutable structure like a list, or a dictionary, has changed.

```
from persistent import Persistent

class Track(Persistent):
    def __init__(self, title, artist=None, albums=[]):
        self.title = title
        self.artist = artist
        self.albums = albums

    def add_album(self, album):
        self.albums.append(album)
        self._p_changed = 1

class Album(Persistent):
    def __init__(self, name, year=None):
        self.name = name
        self.year = year
        self.tracks = []

    def add_track(self, track):
        self.tracks.append(track)
        self._p_changed = 1
```

Let us create a simple script, `store_data.py`, to add some tracks and an album.

```
from app_db import app_db
from app_models import Album, Track
import transaction

db = app_db()
albums = db.droot['Albums']
tracks = db.droot['Tracks']
tracks['Blowing in the Wind'] = Track('Blowing in the Wind', artist='Bob Dylan')
tracks['Like a Rolling Stone'] = Track('Like a Rolling Stone', artist='Bob Dylan')
# the key can be any unique id
albums['UI'] = Album('Ultimate Collection')
# add relationships
album = albums['UI']
track = tracks['Blowing in the Wind']
album.add_album(album)
album.add_track(track)
track = tracks['Like a Rolling Stone']
track.add_album(album)
album.add_track(track)
```

```
transaction.commit()
db.close()
```

Finally, print the data, to see how to access the data in ZODB. Iterate over each album and each track, and print the values of the object. The `details` flag is used to prevent an indefinite recursive loop.

```
from app_db import app_db
from app_models import Track, Album

def print_album(album, details=True):
    print('Name: %s in %s' % (album.name, album.year))
    if details:
        for track in album.tracks:
            print_track(track, details=False)
        print('')

def print_track(track, details=True):
    print('Title: %s by %s' % (track.title, track.artist))
    if details:
        for album in track.albums:
            print_album(album, details=False)
        print('')

db = app_db()
# iterate over albums and tracks
print('List of Albums')
for album in db.droot['Albums'].values():
    print_album(album)
print('List of Tracks')
for track in db.droot['Tracks'].values():
    print_track(track)
db.close()
```

Working with ZODB is almost as easy as dealing with dictionaries. You can use the Python method `isinstance()` to determine the type of an object you are dealing with, and write very versatile and flexible code. ZODB has been around for over a decade, and has been used in various production environments, though the Zope community does not seem to have been successful in marketing it to developers for use outside the Zope (or Plone) environments. **END** 

By: Anil Seth

The author currently works as a consultant, prior to which he was a professor at Padre Conceicao College of Engineering (PCCOE) in Goa. He has also managed IT and imaging solutions for Phil Corporation Limited (Goa), supported domestic customers for Tata Burroughs/TIL, and was a researcher with IIT-K and the Indian Institute of Geomagnetism (Mumbai). You can find him online at <http://sethanil.com> and reach him via email at anil@sethanil.com.